PYTHON ACTIVITY-BASED LEARNING

10 Selected Case Studies

Name	Faisalkhan
Roll Number	26000849
Branch / Section	CSM-1
Subject	Python Programming

Selected Case Studies :

These ten case studies were selected for practice with variables, input/output, conditions, loops, strings, lists, nested lists, tuples, tuple operations, and unpacking, as listed in the uploaded assignment.

Case No	Cases Study	Main concepts
39	Multi-Stage Examination Eligibility System	Input, conditions, lists
53	Food Waste Monitoring System	Loops, arithmetic, conditions
13	ATM Transaction Sequence Simulation	Loops, conditions, running total
51	Bus Passenger Occupancy Simulation	Loops, validation, capacity
28	Repeated Transaction Detection	Lists, membership, conditions

31	String-Based Examination Answer Evaluation	Strings, counters, lists
34	Student Feedback Sentiment Scoring	Strings, split, keyword lists
17	Student Performance Trend Analysis	Nested lists, arithmetic, trends
63	Multi-Product Price Fluctuation Analysis	Lists, percentages, absolute values
49	Tuple-Based Product Catalogue Audit	Tuples, unpacking, totals

Case Study 39: Multi-Stage Examination Eligibility System

1. Problem Statement:

Determine whether a student is eligible for the final examination by checking attendance, laboratory completion, internal assessment marks, and fee clearance. Report every unmet requirement.

2. Objectives:

- Check multiple eligibility conditions.
- Practice statements and logical comparisons.
- Report all conditions that are not satisfied.

3. Assumptions:

- Minimum attendance is 75%.
- Laboratory completion is mandatory.
- Minimum internal assessment mark is 40 out of 100.

• Fees must be marked as paid. These numerical rules are example assumptions because the assignment brief does not give thresholds

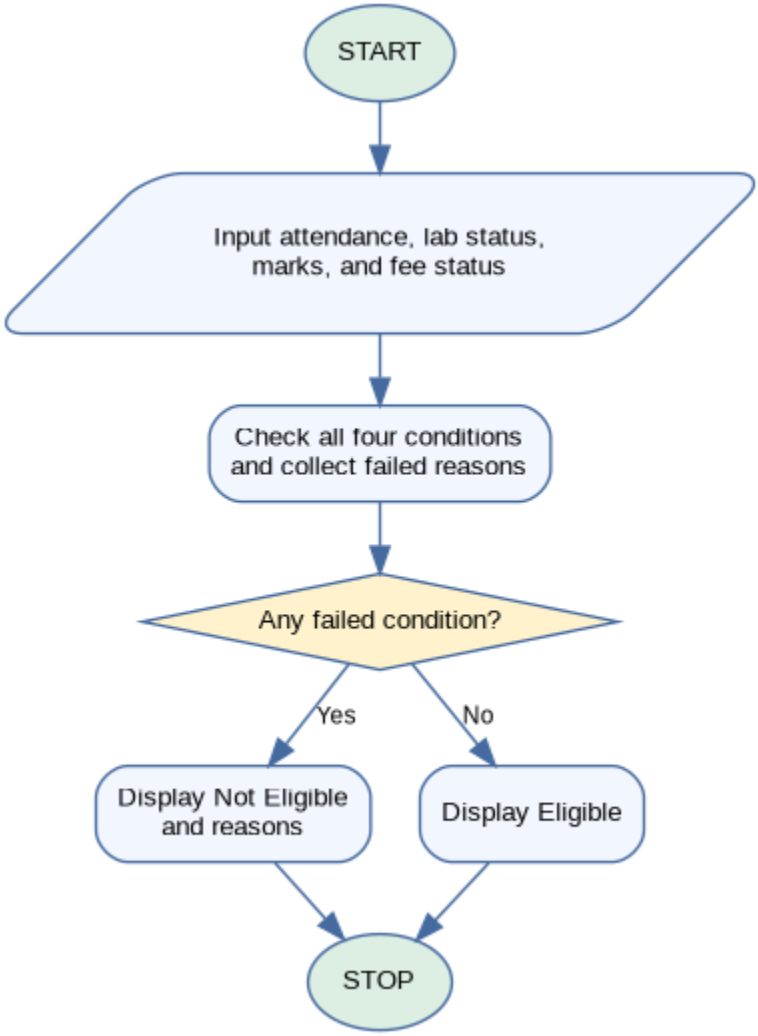
4. Algorithm:

1. Read attendance, laboratory completion, internal marks, and fee status.

2. Create an empty list for unmet requirements.

3. Check each condition and add a reason when it fails. 4. If no reasons were recorded, display eligible; otherwise, display not eligible and each reason.

5. Flowchart (major logic):



6. Pseudocode :

START

INPUT attendance, lab status, marks, fee status

reasons = empty list

IF attendance < 75: add attendance reason

IF lab status is not "yes": add lab reason

IF marks < 40: add marks reason

IF fee status is not "yes": add fee reason

IF reasons is empty: DISPLAY Eligible

ELSE: DISPLAY Not Eligible and all reasons

STOP

Case Study 39: Python Source Code

7. Python Source Code:

```
attendance = float(input("Enter attendance percentage: "))
lab = input("Lab completed? (yes/no): ").lower()
marks = float(input("Enter internal marks: "))
fees = input("Fees paid? (yes/no): ").lower()
reasons = []
if attendance < 75:
    reasons. Append("Attendance below 75%")
if lab != "yes":
    reasons. Append("Lab not completed")\
if marks < 40:
    reasons. Append("Internal marks below 40")
if fees != "yes":
    reasons. Append("Fees not paid")
if Len(reasons) == 0:
print("Eligible for Final Examination")
```

else:

print("Not Eligible")

for reason in reasons:

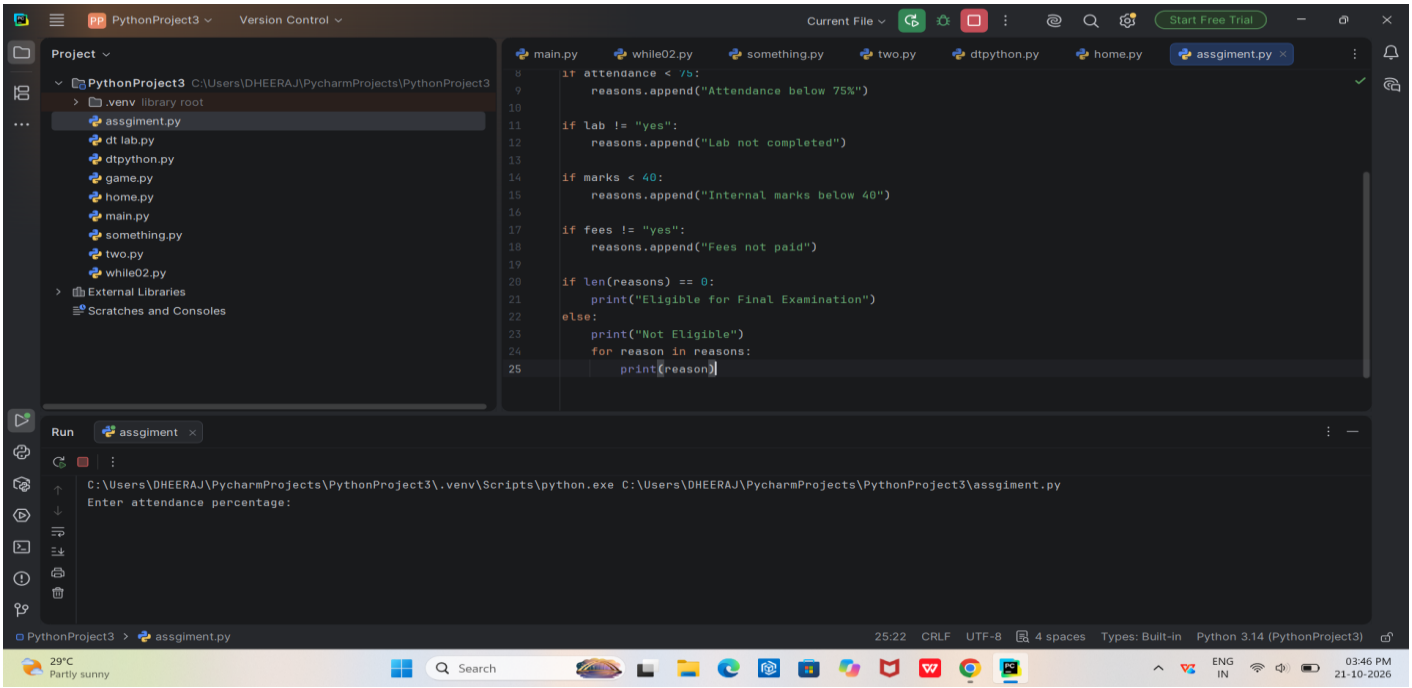
print(reason)

8. Test Case:

Attendance 80%, lab completed yes, internal marks 65, fees paid yes.

Expected result: Eligible for Final Examination

9. Sample Output Capture:



10. Conclusion:

The program checks four requirements and displays eligibility or every unmet requirement.

Case Study 53: Food Waste Monitoring System

1. Problem Statement:

For each day, compare meals prepared with meals served, calculate the number and percentage of meals wasted, and flag high wastage.

2. Objectives:

- Use a loop to process several days.
- Calculate waste and percentage using arithmetic operators.
- Classify waste according to a threshold.

3. Assumptions:

- Prepared meals must be greater than zero.
- Served meals must be between zero and prepared meals.
- Wastage above 20% is treated as high for this example; the source brief does not specify a threshold.

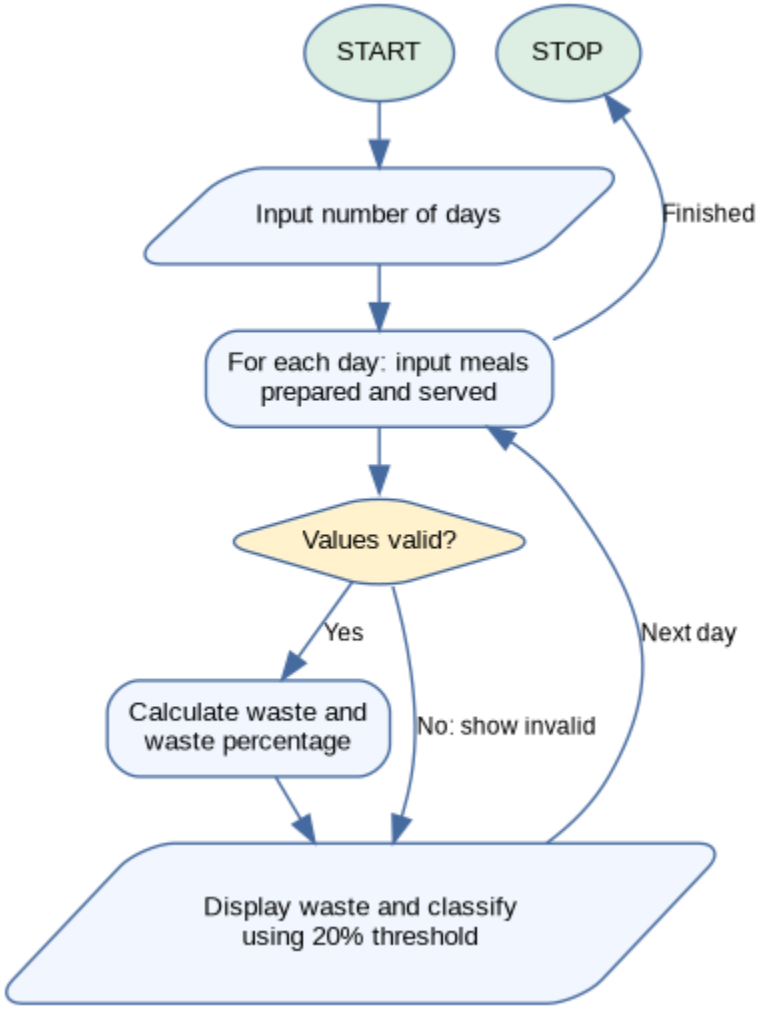
4. Algorithm:

1. Read the number of days.
2. Repeat for each day and read meals prepared and served.
3. Reject invalid values for that day.

4. Calculate waste = prepared - served and waste percentage.

5. Display the waste and classify it using the 20% threshold.

5. **Flowchart (major logic):**



6. Pseudocode:

START

INPUT number_of_days

```
FOR each day
    INPUT prepared, served
IF prepared <= 0 OR served < 0 OR served > prepared
   DISPLAY Invalid values; continue to next day
ELSE
    waste = prepared – served
     percent = waste / prepared * 100
     DISPLAY waste and percent
      IF percent > 20: DISPLAY High Food Waste
      ELSE: DISPLAY Acceptable Food Waste
STOP
```

Case Study 53: Python Source Code

7. Python Source Code:

```
days = int(input("Enter number of days: "))
 for day in range(1, days + 1):
       print("Day", day)
prepared = int(input("Meals prepared: "))
served = int(input("Meals served: "))
if prepared <= 0 or served < 0 or served > prepared:
           print("Invalid values")
            continue waste = prepared – served
percentage = waste / prepared * 100
print("Meals wasted:", waste)
print("Wastage percentage:", percentage, "%")
if percentage > 20: print("High Food Waste")
```

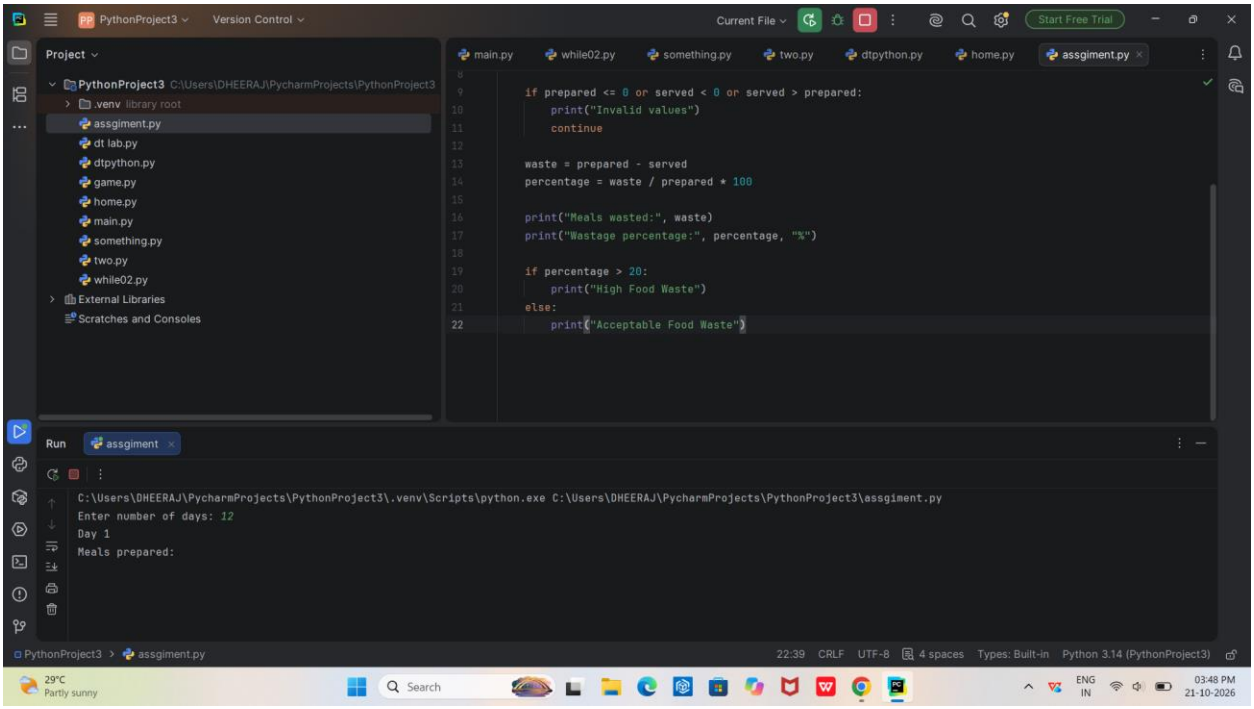
```
else:
    print("Acceptable Food Waste")
```

8. Test Case:

Day 1: 100 prepared, 75 served. Day 2: 50 prepared, 30 served.

Expected result: Day 1 waste is 25 (25%); Day 2 waste is 20 (40%). Both are classified as high waste.

9. Sample Output Capture:



10. Conclusion :

The program calculates daily waste and identifies days that cross the example wastage threshold.

Case Study 13: ATM Transaction Sequence Simulation

1. Problem Statement Simulate a sequence of cash deposits and withdrawals, update the account balance, reject invalid amounts, and prevent withdrawals beyond the available balance.

2. Objectives :

• Practise loops and conditional statements.

• Update a running balance after transactions.

• Reject unsupported transaction types and invalid amounts. 3. Assumptions

• Only deposit and withdraw are supported.

• A withdrawal cannot exceed the current balance.

• No bank fees or withdrawal limits are applied in this example.

4. Algorithm:

1. Read the initial balance and number of transactions.

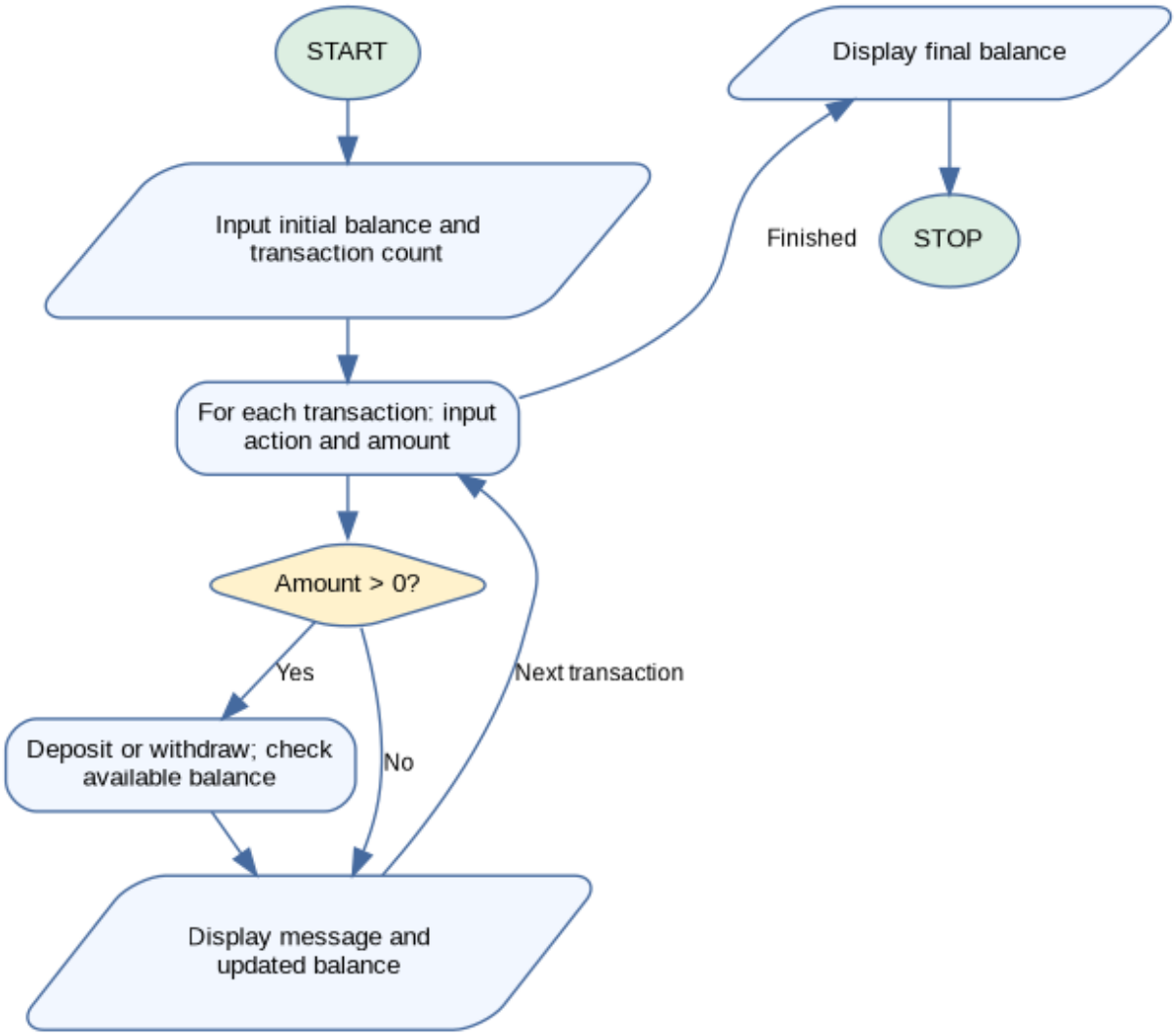
2. For each transaction, read the action and amount.

3. Reject amounts less than or equal to zero.

4. For a deposit, add the amount; for a withdrawal, subtract only if sufficient balance exists.

5. Display the balance after each transaction and at the end.

5. Flowchart (major logic):



6. Pseudocode:

START INPUT balance, number_of_transactions

REPEAT for each transaction

    INPUT action, amount

    IF amount <= 0: DISPLAY Invalid amount

    ELSE IF action is deposit: add amount to balance

    ELSE IF action is withdraw

        IF amount <= balance: subtract amount

        ELSE: DISPLAY Insufficient balance

ELSE: DISPLAY Invalid transaction DISPLAY current balance DISPLAY final balance

STOP

Case Study 13: Python Source Code:

7. Python Source Code:

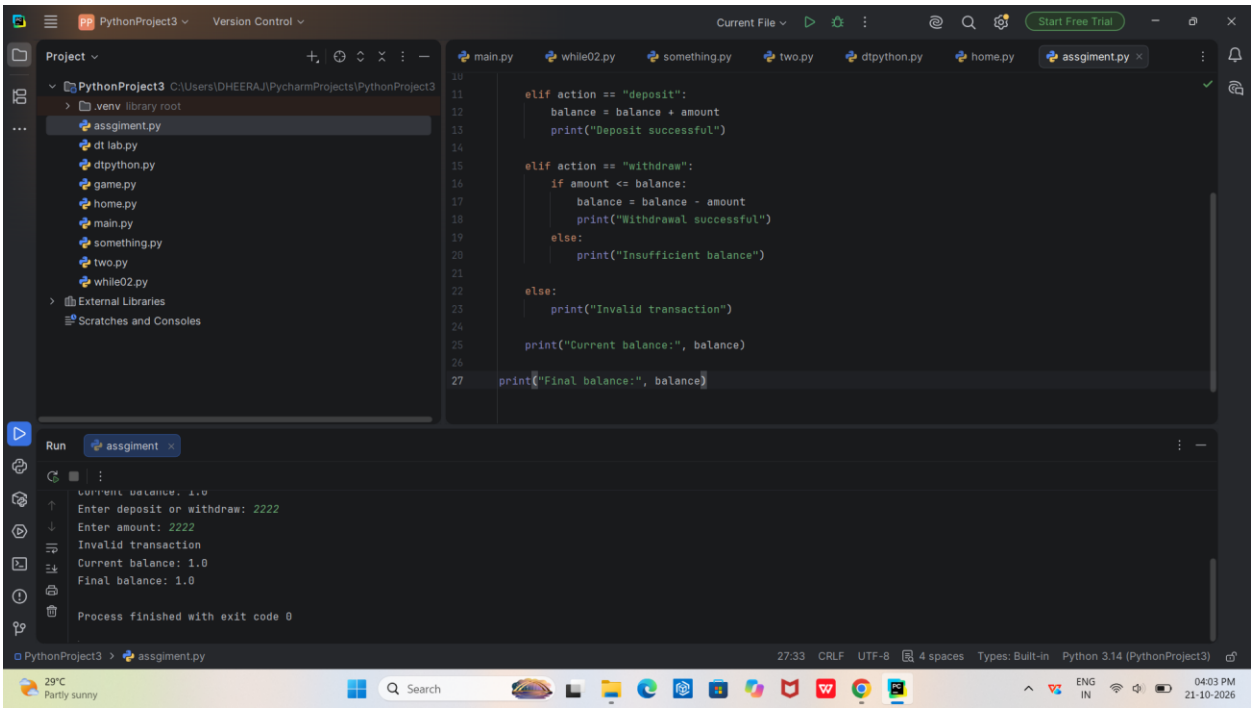
```
balance = float(input("Enter initial balance: "))
 n = int(input("Enter number of transactions: "))
for i in range(n):
        action = input("Enter deposit or withdraw: ").lower()
        amount = float(input("Enter amount: "))
if amount <= 0:
        print("Invalid amount")
 elif action == "deposit":
        balance = balance + amount
        print("Deposit successful")
elif action == "withdraw":
     if amount <= balance:
        balance = balance - amount
       print("Withdrawal successful")
else:
      print("Insufficient balance")
Else:   print("Invalid transaction")
```

```
    print("Current balance:", balance)
print("Final balance:", balance)
```

8. Test Case Initial balance 1000:

Deposit 500, withdraw 200, then attempt to withdraw 2000. Expected result: Balances after transactions: 1500, 1300, 1300. Final balance is 1300.

9. Sample Output Capture:



10. Conclusion:

The program maintains a balance and blocks a withdrawal when funds are insufficient.

Case Study 51: Bus Passenger Occupancy Simulation

1. Problem Statement Track the number of passengers on a bus across several stops, using boarding and alighting counts and checking that the bus capacity is not exceeded.

2. **Objectives:**

- Update passenger counts using arithmetic.
- Validate alighting and boarding values.
- Detect when a boarding request would exceed capacity.

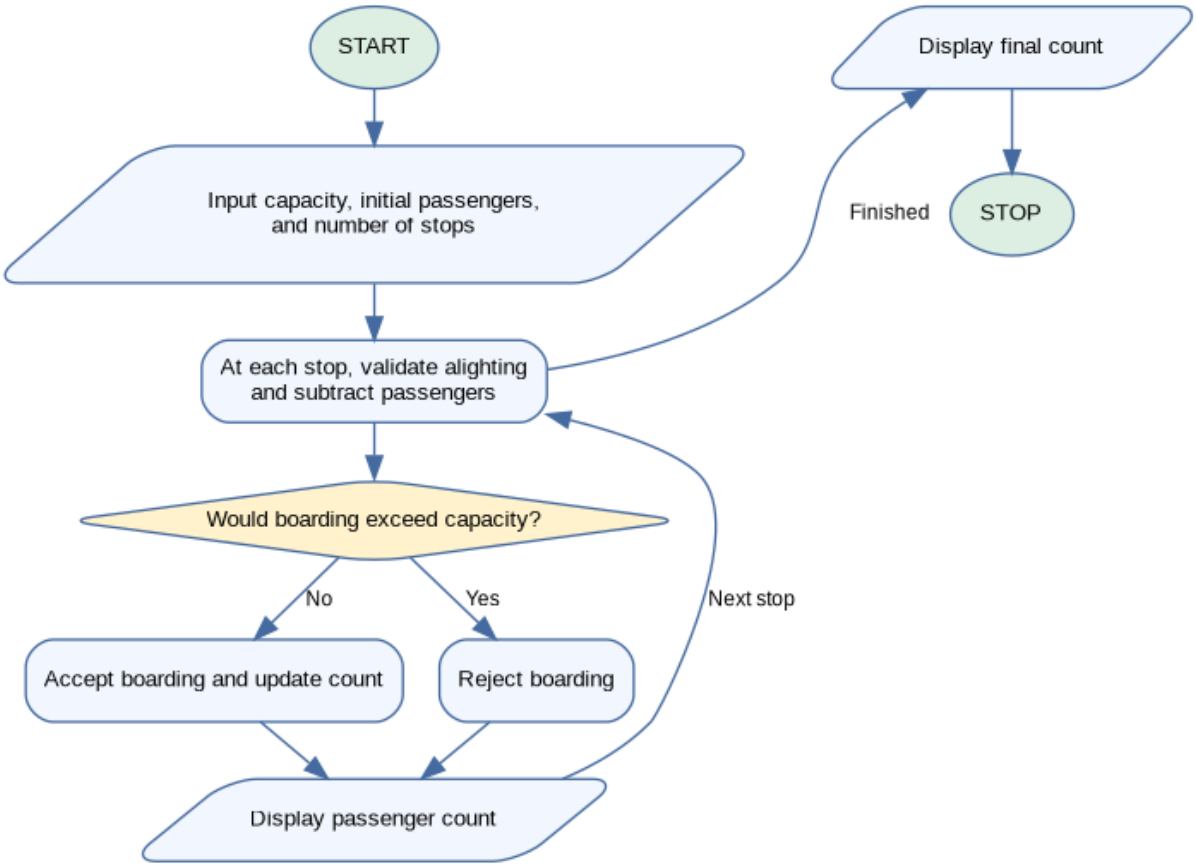
3. Assumptions :

- Passengers and capacity are whole numbers.
- If the alighting count is invalid, the program treats it as zero.
- If boarding would exceed capacity, the boarding request is rejected.

4. Algorithm :

1. Read capacity, initial passengers, and number of stops.
2. At each stop, read the number alighting and validate it.
3. Subtract valid alighting passengers.
4. Read boarding count and reject a negative count or a count that exceeds capacity. 5. Display passengers after each stop and at the end.

5. Flowchart (major logic):



6. Pseudocode:

START

INPUT capacity, passengers, number_of_stops

FOR each stop

INPUT alighting count

IF invalid alighting count: DISPLAY error; set count to 0

passengers = passengers - alighting count

INPUT boarding count

IF boarding count < 0: DISPLAY error

ELSE IF passengers + boarding count > capacity: reject boarding

ELSE: passengers = passengers + boarding count

DISPLAY passengers

DISPLAY final passenger count

STOP

Case Study 51: Python Source Code

7. Python Source Code :

```
capacity = int(input("Enter bus capacity: "))
passengers = int(input("Enter initial passengers: "))
 stops = int(input("Enter number of stops: "))
for i in range(1, stops + 1):
       print("Stop", i)
alight = int(input("Passengers getting down: "))
if alight < 0 or alight > passengers:
     print("Invalid alighting count")
      Alight = 0
passengers = passengers – alight
 board = int(input("Passengers getting in: "))
 if board < 0:
      print("Invalid boarding count")
elif passengers + board > capacity:
print("Capacity exceeded! Boarding rejected")
else:
      passengers = passengers + board
```

print("Passengers on bus:", passengers)

print("Final passenger count:", passengers)

8. Test Case:

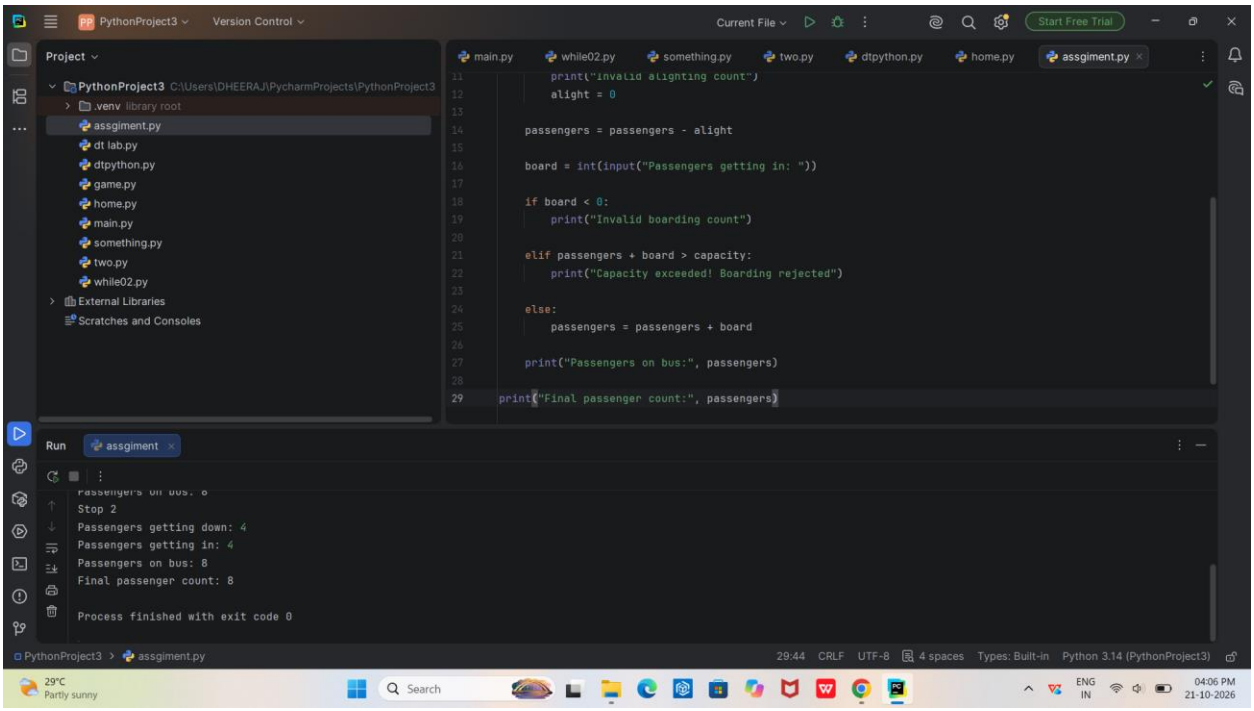
Capacity 40, initial passengers 20.

Stop 1: 5 alight, 10 board. Stop 2: 2 alight, 50 attempt to board.

Expected result:

After stop 1 there are 25 passengers. At stop 2, boarding is rejected and 23 remain.

9. Sample Output Capture:



10. Conclusion:

The program updates passenger occupancy and rejects boarding that would exceed bus capacity.

Case Study 28: Repeated Transaction Detection

1. Problem Statement Read transaction identifiers and amounts, detect repeated identifiers, flag amounts above a threshold, and keep track of transaction identifiers in a list.

2. Objectives:

- Practise list membership checks.
- Use a loop to process multiple transactions.
- Flag duplicates and high-value transactions.

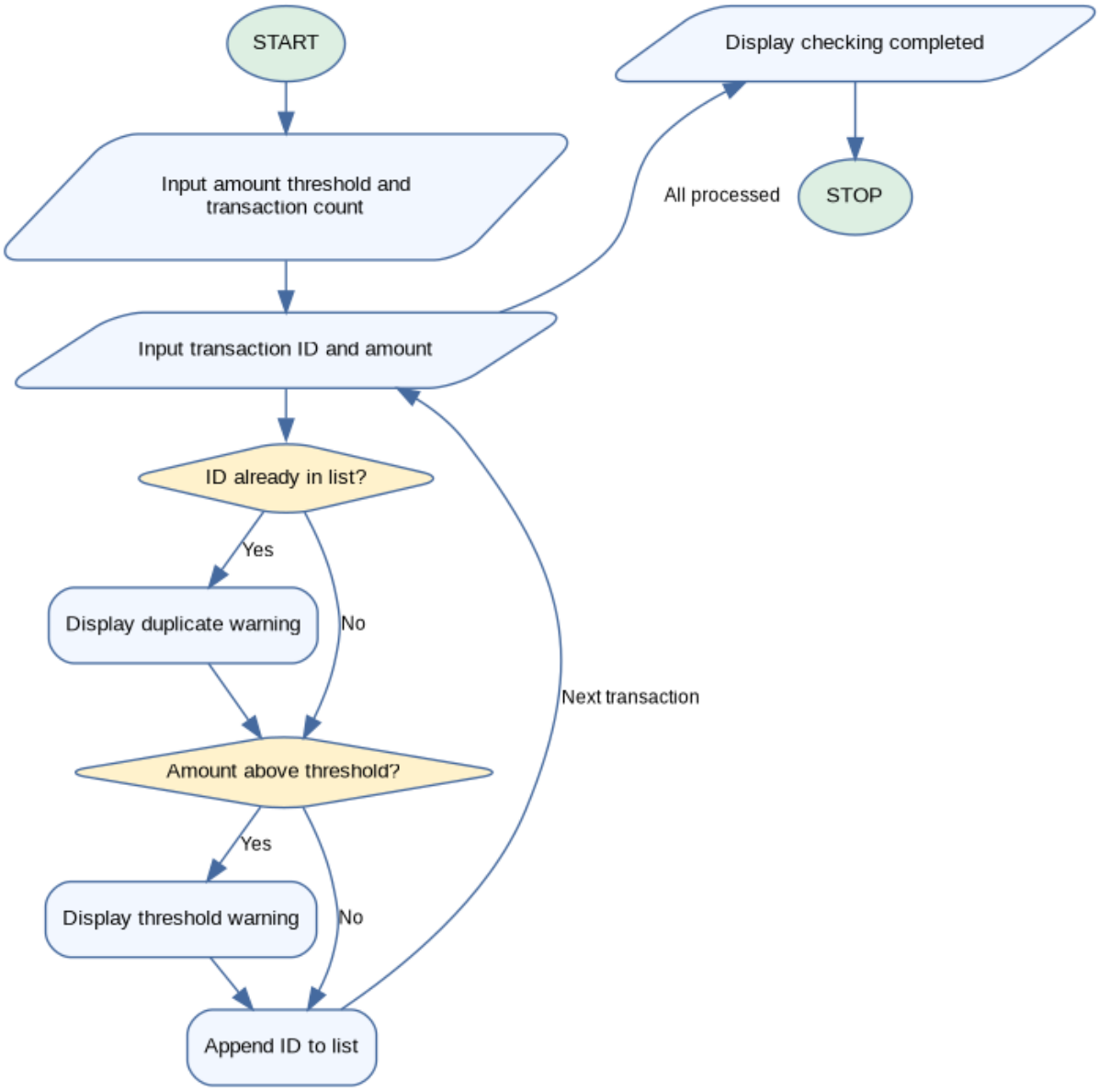
3. Assumptions:

- Transaction IDs are compared exactly as entered.
- An amount is flagged when it is greater than the entered threshold.
- This basic program does not compare transaction timestamps.

4. Algorithm:

1. Create an empty list for transaction IDs.
2. Read the amount threshold and transaction count.
3. For each transaction, read its ID and amount.
4. If the ID already exists, display a duplicate warning.
5. If the amount exceeds the threshold, display a threshold warning.
6. Append the ID and finish after all transactions.

5. Flowchart (major logic):



6. Pseudocode:

START

ids = empty list

INPUT threshold, transaction_count

FOR each transaction

    INPUT transaction_id, amount

    IF transaction_id in ids: DISPLAY duplicate ID

IF amount > threshold: DISPLAY threshold warning

APPEND transaction_id to ids

DISPLAY checking completed

STOP

Case Study 28: Python Source Code

7. Python Source Code:

```
ids = []
threshold = float(input("Enter amount threshold: "))
n = int(input("Enter number of transactions: "))
for i in range(n):
    transaction_id = input("Enter transaction ID: ")
    amount = float(input("Enter transaction amount: "))
if transaction_id in ids:
    print("Duplicate transaction ID:", transaction_id)
if amount > threshold:
    print("Amount exceeds threshold:", amount)
ids.append(transaction_id)
print("Transaction checking completed")
```

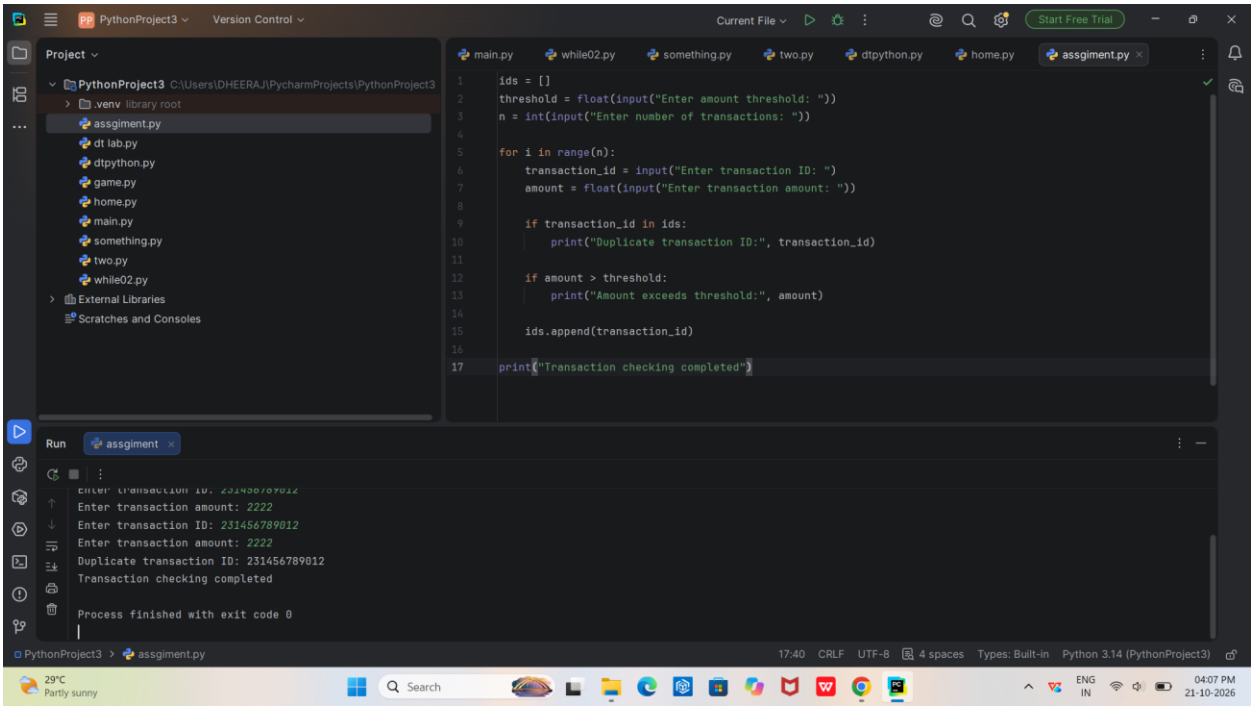
8. Test Case:

Threshold 1000. T01 amount 500, T02 amount 1200, and T02 amount 1500.

Expected result:

The second transaction exceeds the threshold; the third has a duplicate ID and exceeds the threshold.

9. Sample Output Capture:



10. Conclusion:

The program uses list membership to identify repeated IDs and flags amounts above the chosen threshold.

Case Study 31: String-Based Examination Answer Evaluation

1. Problem Statement Compare a student answer for each question with a fixed answer key, count correct, incorrect, and unanswered responses, and calculate a score with negative marking.

2. Objectives :

• Compare strings using conditional statements.

• Count answers in three categories.

• Calculate a final score using arithmetic.

3. Assumptions :

• The example has five questions and a fixed answer key.

• A correct answer receives +1, an incorrect answer receives -0.25, and a blank answer receives 0.

• Answers are converted to uppercase before comparison.

4. Algorithm:

1. Set the answer key and initialize all counters to zero.

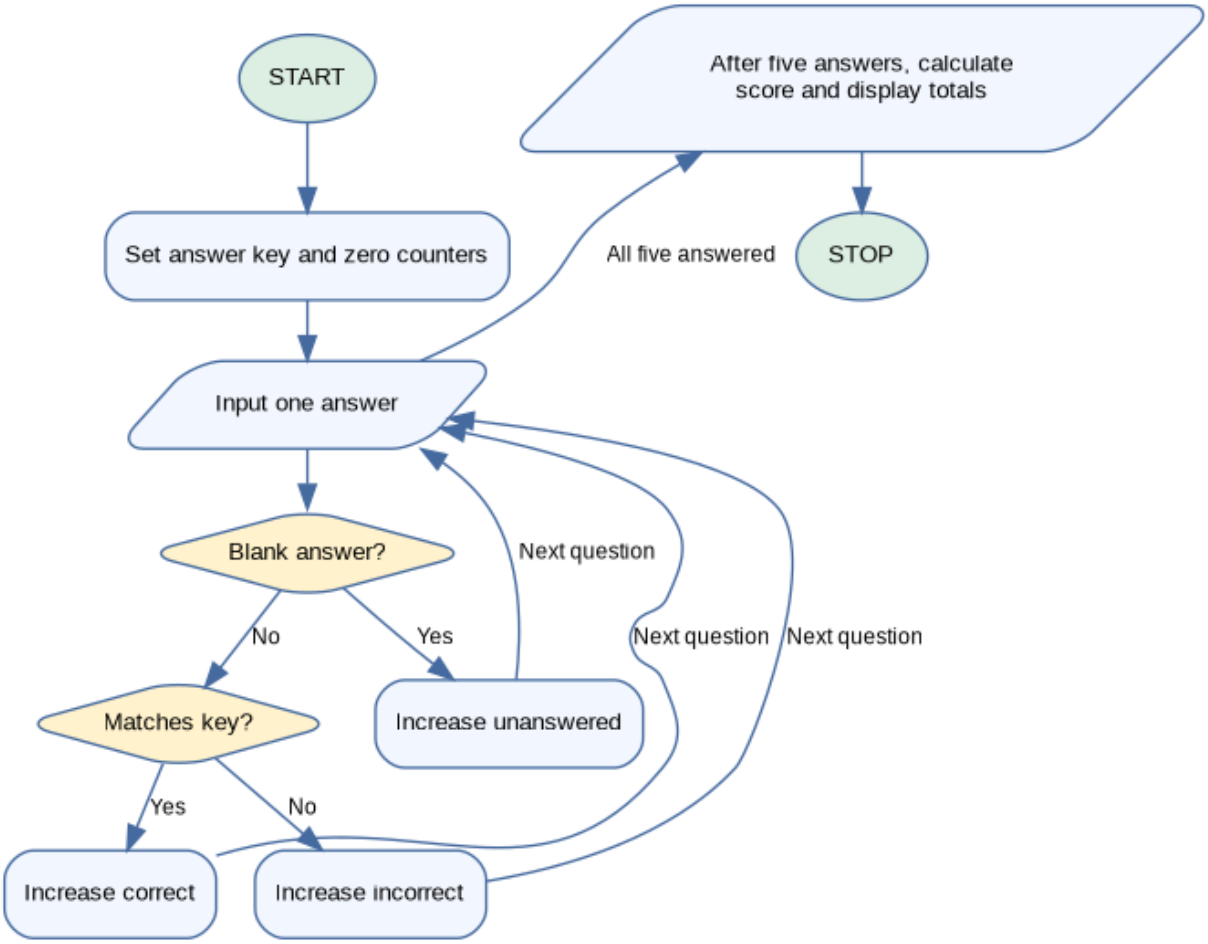
2. Read an answer for each question.

3. Increase the unanswered, correct, or incorrect counter as appropriate.

4. Calculate score = correct - 0.25 times incorrect.

5. Display all counts and the final score.

5. Flowchart (major logic):



6. Pseudocode:

START

key = [A, B, C, D, A]

correct = incorrect = unanswered = 0

FOR each of five questions

    INPUT answer and convert to uppercase IF answer is blank: unanswered += 1

    ELSE IF answer equals key: correct += 1

    ELSE: incorrect += 1

score = correct - 0.25 * incorrect

DISPLAY counts and score out of 5

STOP

Case Study 31: Python Source Code

7. Python Source Code:

```
 key = ["A", "B", "C", "D", "A"]
correct = 0
incorrect = 0
 unanswered = 0
for i in range(5):
 answer = input("Enter answer for question " + str(i + 1) + ": ").upper()
if answer == "":
unanswered = unanswered + 1
    elif answer == key[i]:
 correct = correct + 1
else:
    incorrect = incorrect + 1
score = correct - incorrect * 0.25
 print("Correct answers:", correct)
print("Incorrect answers:", incorrect)
print("Unanswered:", unanswered)
 print("Final score:", score, "out of 5")
```

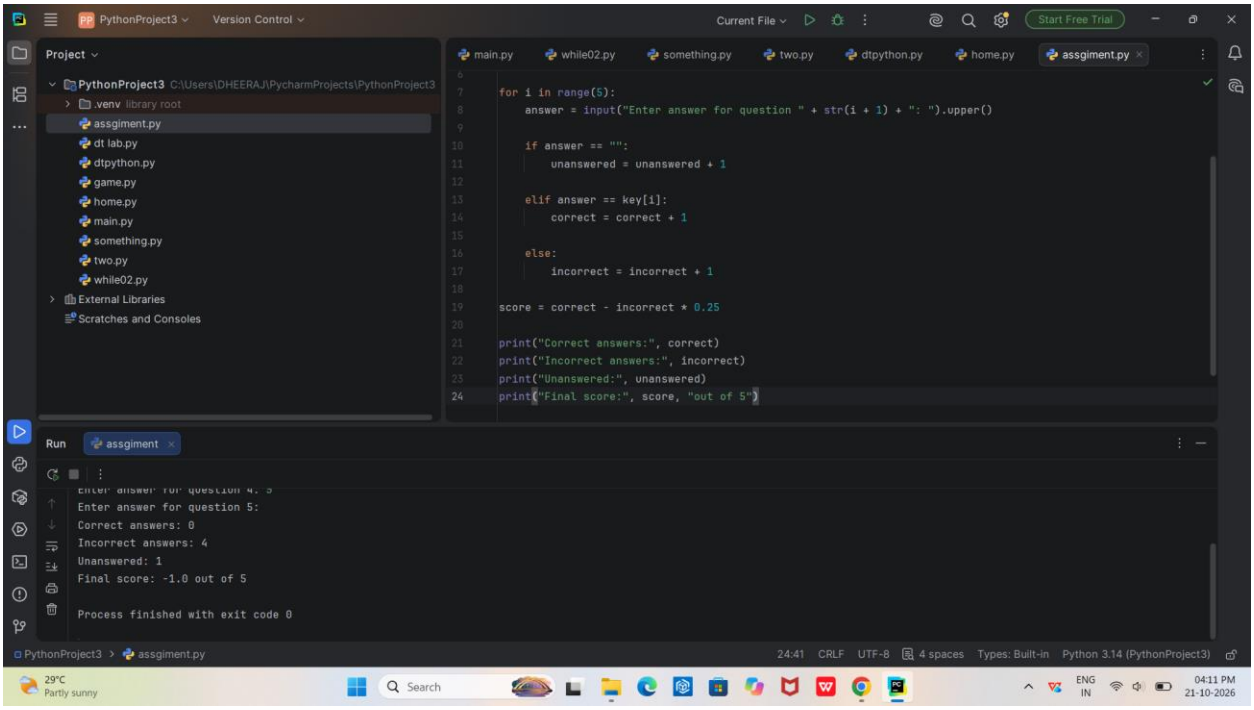
8. Test Case:

Answers are A, B, blank, X, A. The example answer key is A, B, C, D, A.

Expected result:

3 correct, 1 incorrect, 1 unanswered; score = 3 - 0.25 = 2.75 out of 5.

9. Sample Output Capture:



10. Conclusion:

The program evaluates answer strings, counts response categories, and applies the example negative-marking rule.

Case Study 34: Student Feedback Sentiment Scoring

1. Problem Statement Analyze feedback sentences using a supplied list of positive and negative keywords. Count keyword

matches, calculate a score, and classify each feedback sentence.

2. Objectives:

• Use strings and the split method.

• Search lists for matching words.

• Classify feedback using a numeric score.

3. Assumptions:

• Positive and negative words are defined in the program.

• Punctuation is stripped from each word and text is converted to lowercase.

• Score above zero is positive, below zero is negative, and zero is neutral.

4. Algorithm:

1. Define positive and negative keyword lists.

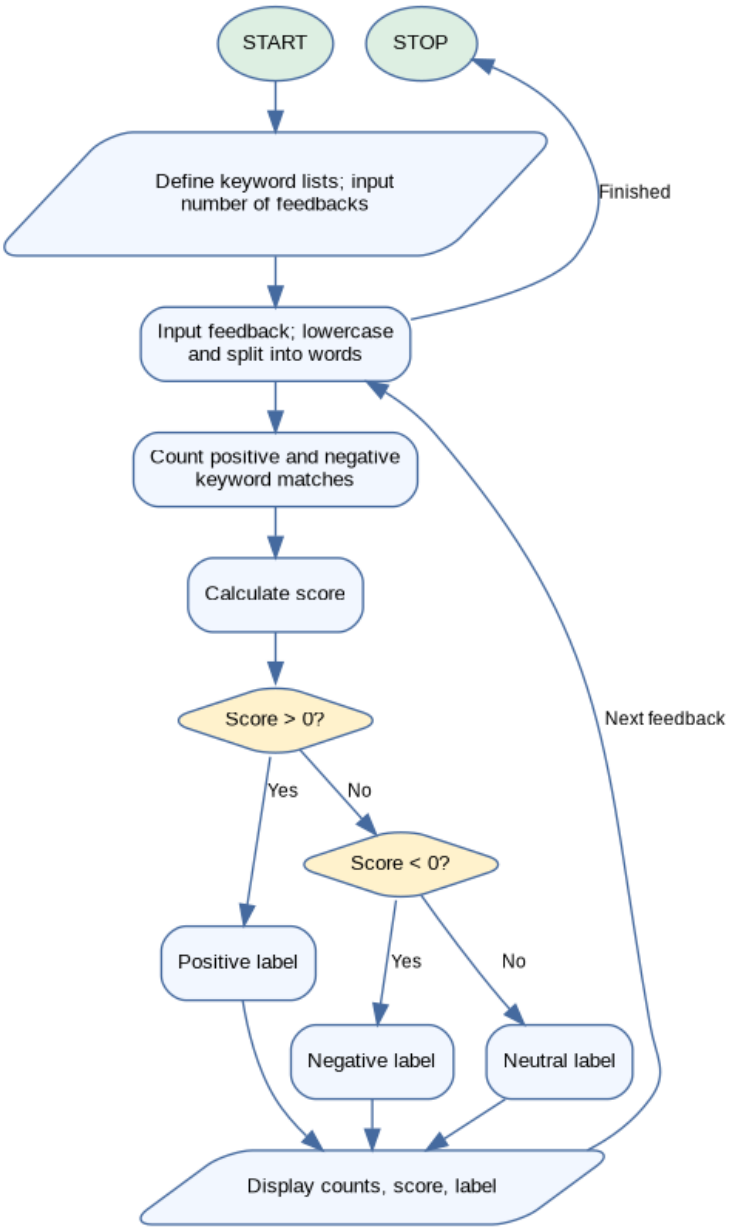
2. Read the number of feedback entries.

3. For each entry, convert it to lowercase and split it into words.

4. Count positive and negative keyword matches.

5. Calculate positive count minus negative count and display the classification.

5. Flowchart (major logic):



6. Pseudocode:

START

Define positive and negative keyword lists

INPUT number_of_feedback_entries

FOR each feedback entry

Convert feedback to lowercase and split into words

Set positive_count and negative_count to zero

FOR each word: remove punctuation and check both lists

score = positive_count - negative_count

IF score > 0: positive; ELSE IF score < 0: negative; ELSE neutral

DISPLAY counts, score, and classification

STOP

Case Study 34: Python Source Code

7. Python Source Code:

```
positive = ["good", "excellent", "helpful", "great", "clear"]
negative = ["bad", "poor", "boring", "confusing", "difficult"]
n = int(input("Enter number of feedbacks: "))
for i in range(n):
    feedback = input("Enter feedback: ").lower()
    words = feedback.split()
pos = 0
neg = 0
for word in words:
    word = word.strip(".,!?;:")
  if word in positive:
```

```
 pos = pos + 1
 if word in negative:
neg = neg + 1 score = pos - neg
print("Positive words:", pos)
print("Negative words:", neg)
 print("Score:", score)
if score > 0:
    print("Positive Feedback")
elif score < 0:
    print("Negative Feedback")
else:
     print("Neutral Feedback")
```

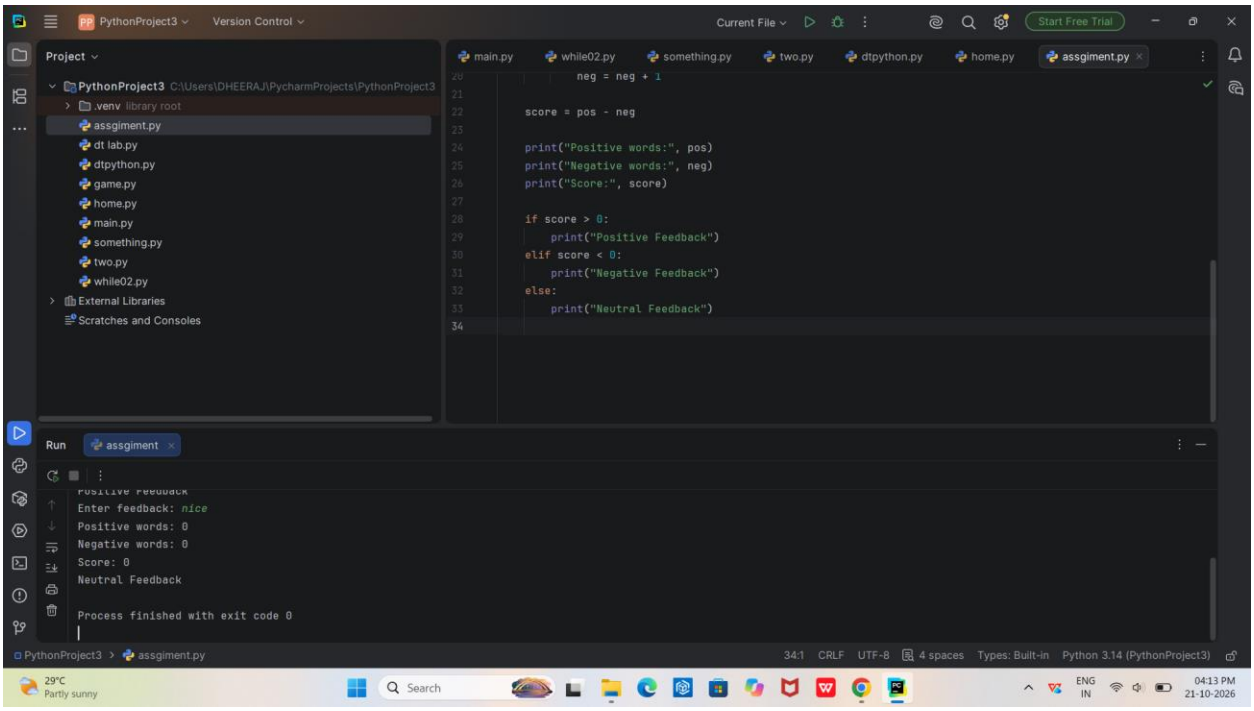
8. Test Case:

Feedback 1 includes good and helpful. Feedback 2 includes boring and confusing.

Expected result:

Feedback 1 score is 2 and is positive. Feedback 2 score is -2 and is negative.

9. Sample Output Capture:



10. Conclusion:

The program uses simple keyword matching to produce a basic positive, negative, or neutral label.

Case Study 17: Student Performance Trend Analysis

1. Problem Statement Compare marks across three examinations for several subjects, calculate the change between the first and third examination, and identify consistent weakness, improvement, or decline.

2. Objectives:

- Store subject names and marks in lists.
- Use a nested list to group exam marks.
- Calculate changes and classify trends.

3. Assumptions:

• Marks are entered on a 0 to 100 scale.

• The program compares Exam 1 with Exam 3 for overall change.

• A subject is consistently weak when all three marks are below 40; these thresholds are examples.

4. Algorithm:

1. Read the number of subjects.

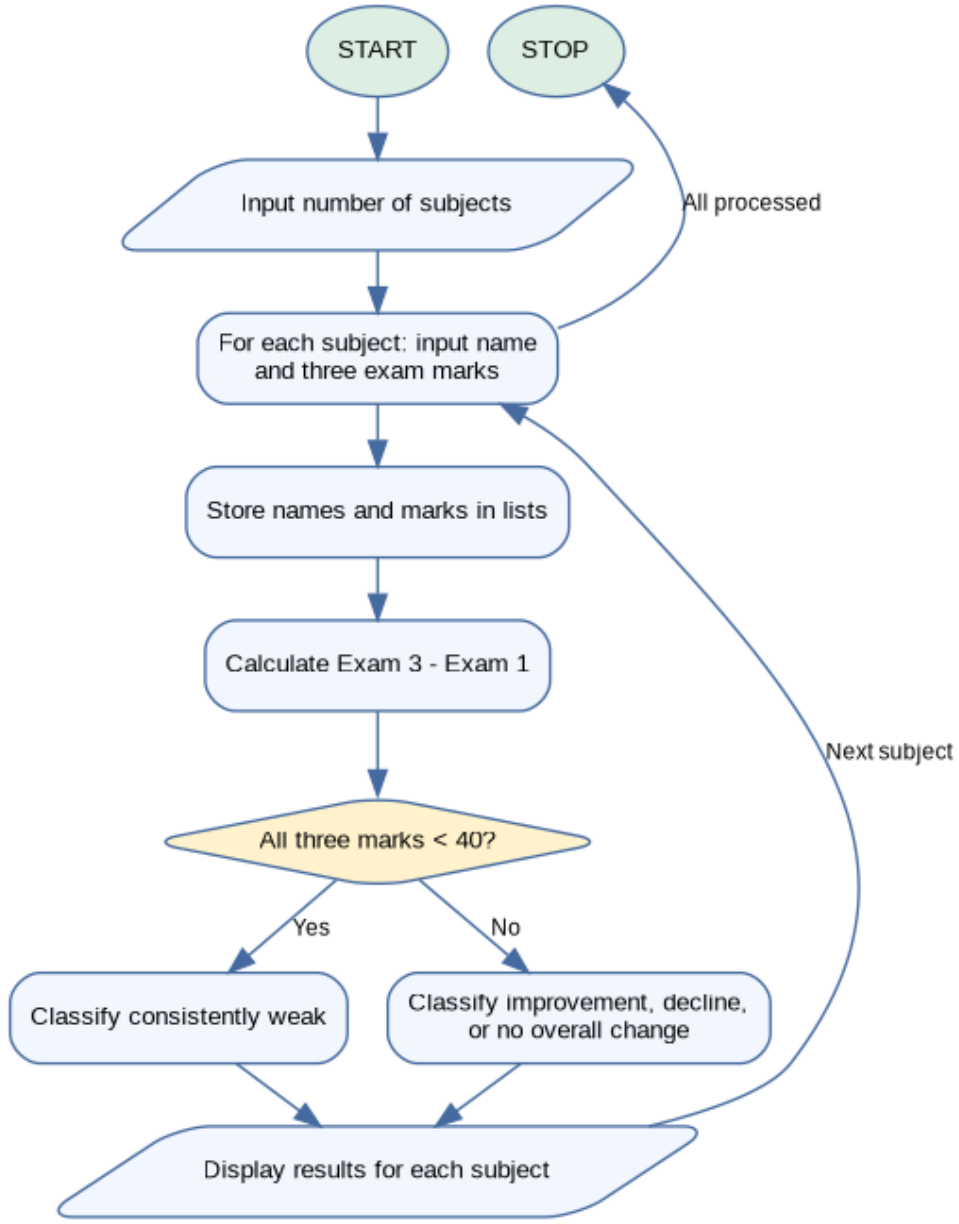
2. For each subject, read its name and three exam marks.

3. Store the subject name and its three marks in lists.

4. For each subject, calculate Exam 3 minus Exam 1.

5. Classify consistently weak subjects first; otherwise show improvement, decline, or no overall change.

5. Flowchart (major logic):



6. Pseudocode:

START

INPUT number_of_subjects

Create subjects list and marks list

FOR each subject: INPUT name, exam1, exam2, exam3

Append name to subjects

Append [exam1, exam2, exam3] to marks

FOR each subject

change = exam3 - exam1

IF all three marks < 40: display consistently weak ELSE IF change > 0: display improved

ELSE IF change < 0: display declined

ELSE: display no overall change

STOP

Case Study 17: Python Source Code :

7. Python Source Code:

```
n = int(input("Enter number of subjects: "))
subjects = []
marks = []
for i in range(n):
    subject = input("Enter subject name: ")
m1 = float(input("Enter exam 1 marks: "))
m2 = float(input("Enter exam 2 marks: "))
m3 = float(input("Enter exam 3 marks: "))
subjects.append(subject)
marks.append([m1, m2, m3])
```

```
for i in range(n):
   print("\nSubject:", subjects[i])
improvement = marks[i][2] - marks[i][0]
print("Change from exam 1 to exam 3:", improvement)

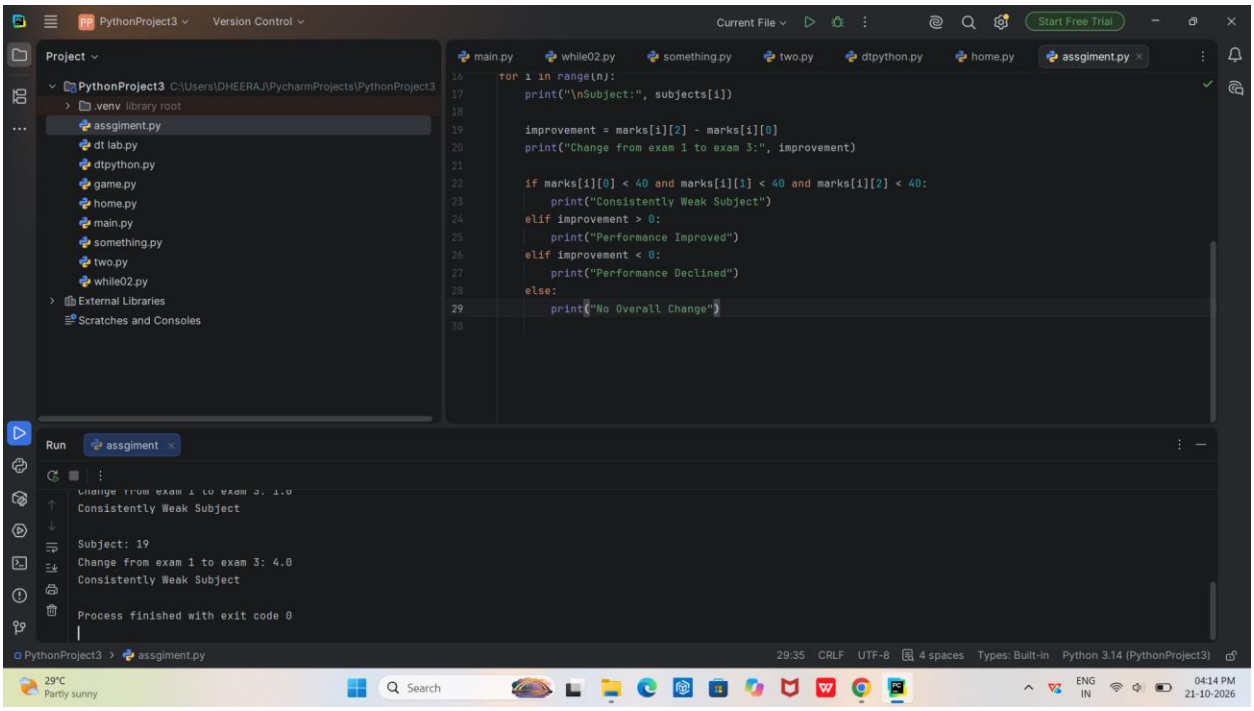

 if marks[i][0] < 40 and marks[i][1] < 40 and marks[i][2] < 40:
     print("Consistently Weak Subject")
 elif improvement > 0:
     print("Performance Improved")
elif improvement < 0:
     print("Performance Declined")
 else:
     print("No Overall Change")
```

8. Test Case:

Math marks are 30, 35, 45. English marks are 60, 55, 50. **Expected result:**

Math changes by +15 and improves. English changes by -10 and declines.

9. Sample Output Capture:



10. Conclusion:

The program stores assessment marks in nested lists and summarizes performance trends for each subject.

Case Study 63: Multi-Product Price Fluctuation Analysis

1. Problem Statement:

Compare previous and current prices for several products, calculate percentage changes, flag changes above a threshold, and identify the product with the largest absolute fluctuation.

2. Objectives:

- Use lists to store product names and percentage changes.
- Calculate percentage changes with arithmetic operators.
- Identify the largest absolute fluctuation.

3. Assumptions:

• Old prices must be positive.

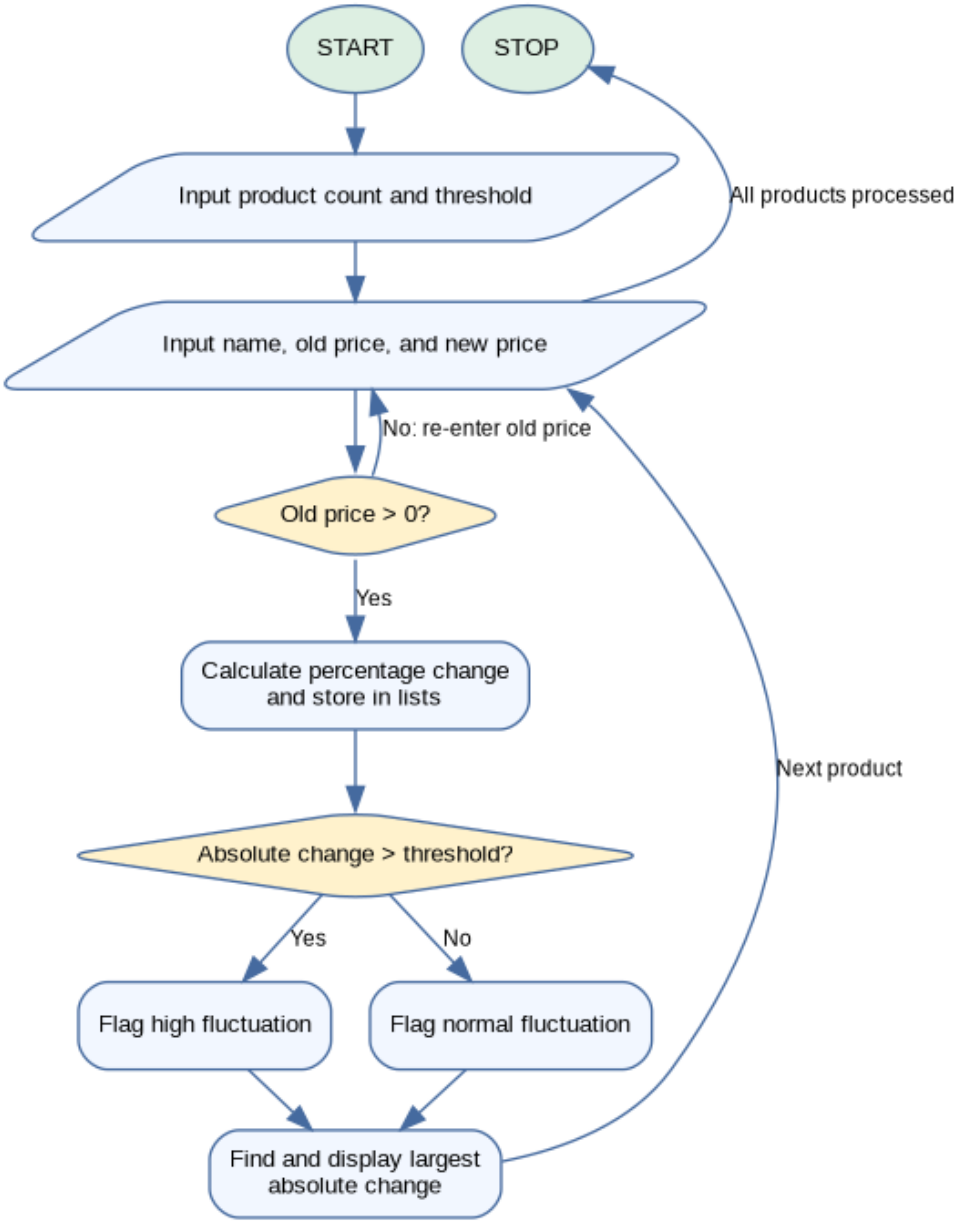
• The example compares absolute percentage change with the threshold.

• New prices are expected to be zero or positive; the basic version does not separately validate them.

4. Algorithm:

1. Read product count and fluctuation threshold.
2. For each product, read its name, old price, and new price.
3. Repeat input for the old price until it is positive.
4. Calculate percentage change and store the product and change.
5. Flag changes whose absolute value exceeds the threshold.
6. Find and display the largest absolute percentage change.

5. Flowchart (major logic):



6. Pseudocode:

START

INPUT product_count, threshold

Create names and changes lists

FOR each product

INPUT name and old price

WHILE old price <= 0: ask again

INPUT new price change = (new - old) / old * 100

Store name and change; display change

IF absolute change > threshold: display high fluctuation

ELSE: display normal fluctuation

Find index with largest absolute change

IF product list is not empty: display that product and change

STOP

Case Study 63: Python Source Code :

7. Python Source Code:

```
n = int(input("Enter number of products: "))
threshold = float(input("Enter fluctuation threshold (%): "))
names = []
changes = []
for i in range(n):
    name = input("Enter product name: ")
    old_price = float(input("Enter old price: "))
while old_price <= 0:
```

```
        print("Old price must be positive")
        old_price = float(input("Enter old price again: "))
new_price = float(input("Enter new price: "))
 change = (new_price - old_price) / old_price * 100
names.append(name)
 changes.append(change)
 print("Percentage change:", round(change, 2), "%")
if abs(change) > threshold:
        print("High Price Fluctuation")
 else:
        print("Normal Price Fluctuation")
 largest = 0
for i in range(1, len(changes)):
if abs(changes[i]) > abs(changes[largest]):
 largest = i
if n > 0:
    print("Largest fluctuation:", names[largest])
     print("Percentage change:", round(changes[largest], 2), "%")
```

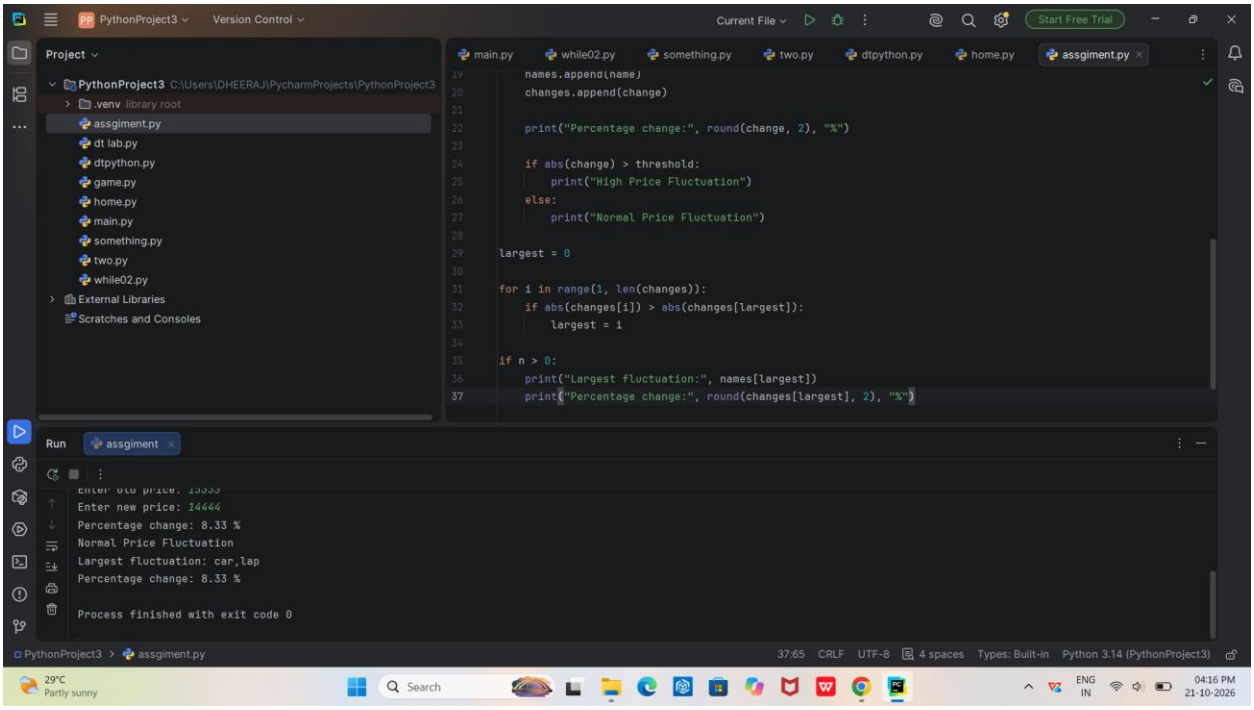
8. Test Case:

Threshold 20%. Laptop 50000 to 55000, Phone 10000 to 12000, Headphones 2000 to 1500.

Expected result:

Changes are 10%, 20%, and -25%. Headphones has the largest absolute fluctuation and is flagged high.

9. Sample Output Capture:



10. Conclusion:

The program calculates price movement percentages and identifies the product with the largest absolute change.

Case Study 49: Tuple-Based Product Catalogue Audit

1. Problem Statement Store product identifiers, prices, and stock quantities in tuples. Validate records, calculate inventory values, and identify products that require replenishment.

2. Objectives:

• Create tuples and use tuple unpacking.

• Validate product prices and quantities.

• Calculate total inventory value and find low-stock items.

3. Assumptions:

• Each record contains a product ID, price, and quantity.

• Negative prices and negative quantities are invalid.

• Stock below the user-entered minimum level needs replenishment.

4. Algorithm:

1. Read the number of products and minimum stock level.

2. Read each product ID, price, and stock quantity and store them as a tuple.

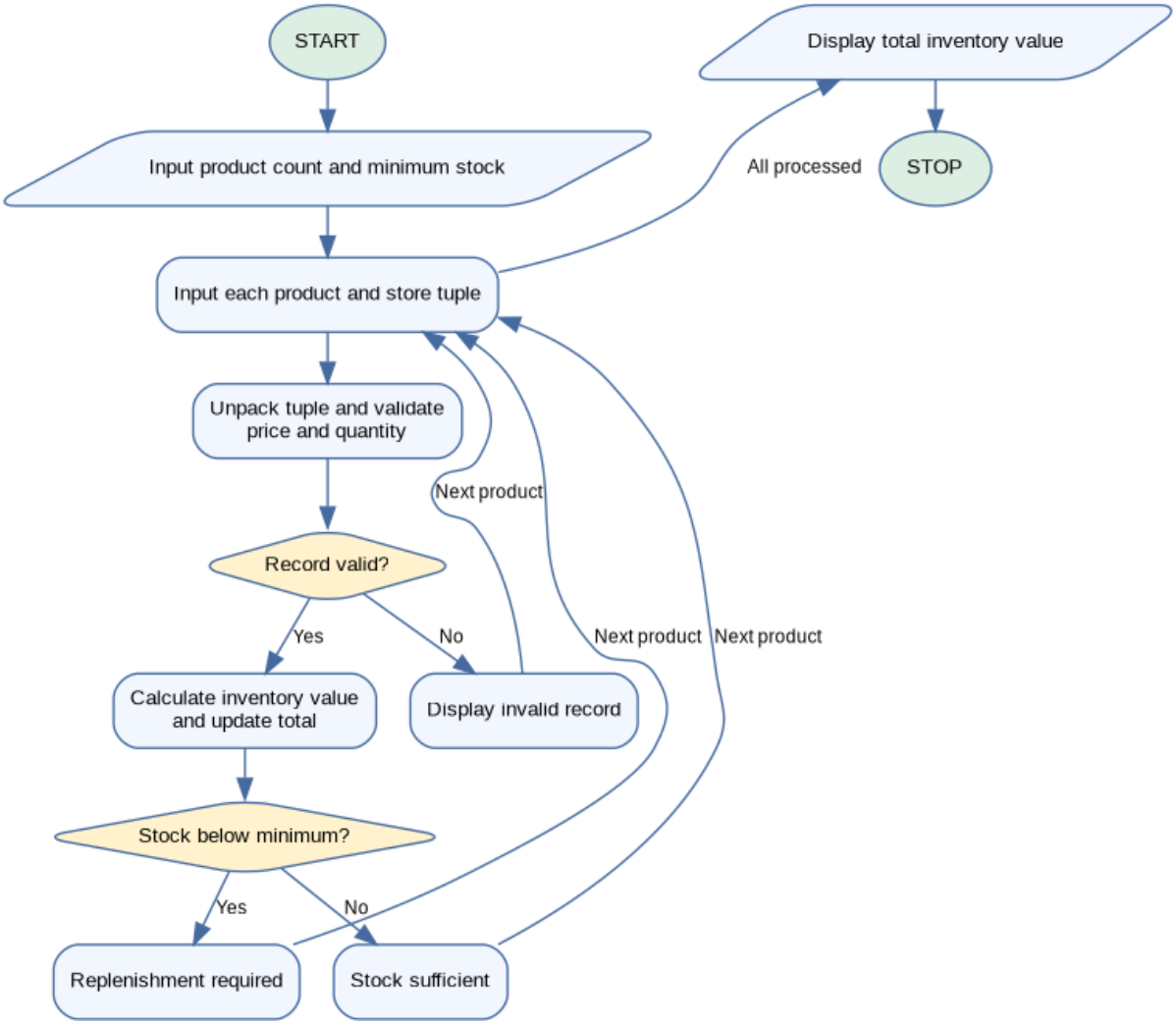
3. Unpack each tuple while processing the catalogue.

4. If price or quantity is negative, report an invalid record.

5. Otherwise, calculate inventory value, update the total, and compare stock with the minimum.

6. Display the total inventory value.

5. Flowchart (major logic):



6. Pseudocode:

START

INPUT product_count, minimum_stock

products = empty list

FOR each product: INPUT ID, price, quantity; append tuple

total_value = 0

FOR each tuple (ID, price, quantity)

IF price < 0 OR quantity < 0: display invalid record

ELSE

value = price * quantity

total_value += value

IF quantity < minimum_stock: display replenishment required

ELSE: display stock sufficient

DISPLAY total inventory value

STOP

Case Study 49: Python Source Code:

7. Python Source Code :

```
products = []
n = int(input("Enter number of products: "))
minimum_stock = int(input("Enter minimum stock level: "))
for i in range(n):
    product_id = input("Enter product ID: ")
    price = float(input("Enter product price: "))
    quantity = int(input("Enter stock quantity: "))
products.append((product_id, price, quantity
total_value = 0
for product_id, price, quantity in products:
    print("\nProduct ID:", product_id)
```

```
if price < 0 or quantity < 0:
        print("Invalid Product Record")
else:
         value = price * quantity
         total_value = total_value + value
print("Price:", price)
 print("Quantity:", quantity)
print("Inventory value:", value)
 if quantity < minimum_stock:
        print("Replenishment Required")
else:
        print("Stock Sufficient")
 print("\nTotal Inventory Value:", total_value)
```

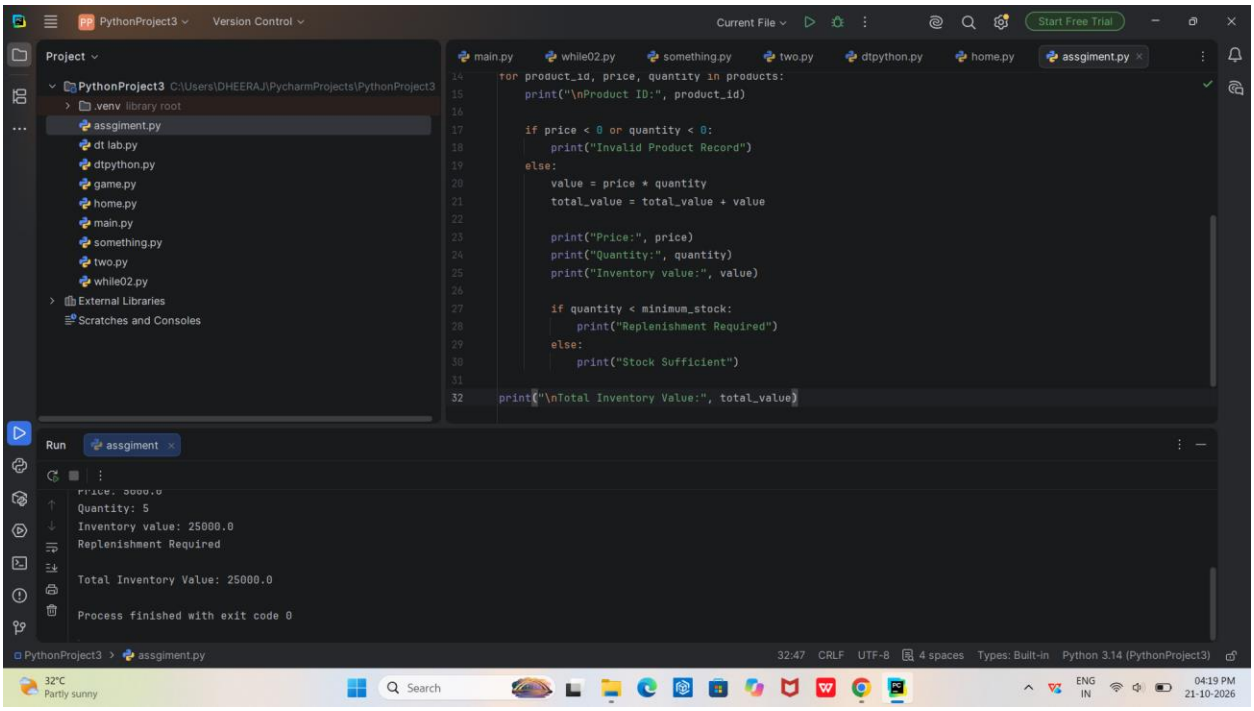
8. Test Case:

Minimum stock 5. P101 price 1000 quantity 10; P102 price 200 quantity 3; P103 price 150 quantity 0.

Expected result:

Inventory values are 10000, 600, and 0. P102 and P103 need replenishment. Total inventory value is 10600.

9. Sample Output Capture:



10. Conclusion:

The program audits a tuple-based catalogue, totals valid inventory value, and flags low-stock products.

THE END...